

Documentação de Consumo da API: TheMealDB

1. Descrição Geral e Funcionalidades da API

A **TheMealDB** é uma API de receitas culinárias internacionais. Para desenvolvimento e uso educacional, a documentação oficial permite a chave de teste **1**. Para publicação em loja de aplicativos, é necessária uma chave de apoiador.

- **Busca de Receitas:** Pesquisa pelo nome do prato ou pela primeira letra.
- **Detalhamento Completo:** Consulta de instruções, categoria, área, país quando informado, foto, fonte, vídeo e ingredientes.
- **Receita Aleatória:** Sugestão aleatória de um prato.
- **Filtros e Classificações:** Filtragem por categoria, área/origem ou um ingrediente principal.
- **Listagem de Metadados:** Listas de categorias, áreas e ingredientes cadastrados.
- **Imagens:** Uso da foto principal da receita e das miniaturas disponibilizadas pela API.

2. Endpoints para Consulta Direta

Principais endpoints. Clique no endereço azul para abrir a resposta JSON no navegador:

Funcionalidade	Método	Endpoint (clique para abrir o JSON)
Buscar receita por nome	GET	https://www.themealdb.com/api/json/v1/1/search.php?s=Arrabiata
Buscar receitas por letra	GET	https://www.themealdb.com/api/json/v1/1/search.php?f=a
Detalhes da receita por ID	GET	https://www.themealdb.com/api/json/v1/1/lookup.php?i=52772

Funcionalidade	Método	Endpoint (clique para abrir o JSON)
Receita Aleatória	GET	https://www.themealdb.com/api/json/v1/1/random.php
Listar todas as Categorias	GET	https://www.themealdb.com/api/json/v1/1/categories.php
Listar nomes de Categorias	GET	https://www.themealdb.com/api/json/v1/1/list.php?c=list
Listar Áreas / Origens	GET	https://www.themealdb.com/api/json/v1/1/list.php?a=list
Listar Ingredientes	GET	https://www.themealdb.com/api/json/v1/1/list.php?i=list
Filtrar por Categoria	GET	https://www.themealdb.com/api/json/v1/1/filter.php?c=Seafood
Filtrar por Área / Origem	GET	https://www.themealdb.com/api/json/v1/1/filter.php?a=Canadian
Filtrar por Ingrediente principal	GET	https://www.themealdb.com/api/json/v1/1/filter.php?i=chicken_breast

Observação: filtro por vários ingredientes, seleção de 10 receitas aleatórias e receitas mais recentes são recursos premium. Eles não são usados pelo MealFinder com a chave educacional.

3. Estruturas JSON Retornadas e Significado dos Campos

Exemplo 1: Detalhes da Receita (/lookup.php?i=52772)

Os endpoints de busca por nome, letra, ID e receita aleatória retornam { "meals": [...] }. A seguir está uma estrutura resumida e atualizada:

```
{
  "meals": [{
    "idMeal": "52772", "strMeal": "Teriyaki Chicken Casserole",
    "strMealAlternate": null, "strCategory": "Chicken",
    "strArea": "Japanese", "strCountry": "Japan",
    "strInstructions": "...", "strMealThumb": "https://...jpg",
    "strTags": "Meat,Casserole", "strYoutube": "https://youtube...",
    "strIngredient1": "Soy Sauce", "strMeasure1": "3/4 cup",
    "strSource": "...", "dateModified": null
  }]
}
```

Dicionário dos Campos da Receita Completa:

Campo	Tipo sugerido	Significado
<code>idMeal</code>	String	Identificador único da receita.
<code>strMeal</code>	String	Nome principal da receita.
<code>strMealAlternate</code>	String?	Nome alternativo, quando houver.
<code>strCategory</code>	String?	Categoria do prato: Chicken, Seafood, Pasta, Dessert etc.
<code>strArea</code>	String?	Área ou tradição culinária: Japanese, Italian, Mexican etc.
<code>strCountry</code>	String?	País associado à área, quando informado.
<code>strInstructions</code>	String?	Texto do modo de preparo.
<code>strMealThumb</code>	String?	URL da imagem principal da receita.
<code>strTags</code>	String?	Tags ou palavras-chave separadas por vírgula.
<code>strYoutube</code>	String?	URL de vídeo no YouTube, quando houver.
<code>strSource</code>	String?	URL da fonte/origem da receita, quando houver.
<code>strImageSource</code>	String?	Crédito ou fonte da imagem, quando houver.
<code>strCreativeCommonsConfirmed</code>	String?	Indicação da API sobre confirmação Creative Commons.
<code>dateModified</code>	String?	Data/hora da última alteração, quando informada.

- **`strIngredient1` até `strIngredient20`:** Nomes dos ingredientes, em até 20 posições.
- **`strMeasure1` até `strMeasure20`:** Quantidade/medida correspondente a cada ingrediente.

Campos opcionais podem retornar **null** ou uma string vazia. Por isso, o Flutter deve utilizar **String?** ou valores padrão ao converter o JSON.

Exemplo 2: Listagens, Categorias e Filtros

Os filtros retornam uma estrutura resumida, ideal para catálogo e cards. A resposta atual dos filtros inclui os seguintes campos:

Campo do filtro	Significado
<code>strMeal</code>	Nome resumido do prato.
<code>strMealThumb</code>	URL da foto da receita.
<code>idMeal</code>	ID usado para consultar o detalhe posteriormente em <code>/lookup.php?i={id}</code> .
<code>strArea</code>	Área/origem da receita, quando informada.
<code>strCountry</code>	País associado à área, quando informado.

Endpoint / objeto	Informações retornadas
<code>categories.php</code> (categories)	<code>idCategory</code> , <code>strCategory</code> , <code>strCategoryThumb</code> , <code>strCategoryDescription</code> .
<code>list.php?c=list</code>	<code>strCategory</code> .
<code>list.php?a=list</code>	<code>strArea</code> e <code>strCountry</code> .
<code>list.php?i=list</code>	<code>idIngredient</code> , <code>strIngredient</code> , <code>strDescription</code> , <code>strThumb</code> e <code>strType</code> .

Os filtros não trazem instruções ou ingredientes. Para obter esses dados, use o **idMeal** retornado no endpoint `/lookup.php?i={id}`.

4. Estruturas de Dados em Dart/Flutter

A. Modelo usado nesta primeira entrega (**Meal**)

Este é o modelo existente no MealFinder. Ele absorve somente os campos usados pela tela inicial:

```
class Meal {
  const Meal({
    required this.id, required this.name, required this.imageUrl,
    required this.category, required this.area,
  });
  final String id;
  final String name;
  final String imageUrl;
  final String category;
  final String area;

  factory Meal.fromJson(Map<String, dynamic> json) => Meal(
    id: json['idMeal'] as String? ?? '',
    name: json['strMeal'] as String? ?? 'Receita sem nome',
    imageUrl: json['strMealThumb'] as String? ?? '',
    category: json['strCategory'] as String? ?? 'Sem categoria',
    area: json['strArea'] as String? ?? 'Origem não informada',
  );
}
```

B. Modelos previstos para as próximas etapas

Estrutura	Dados que deve absorver	Uso futuro
MealDetail	Dados de Meal + instruções, tags, URLs e ingredientes/medidas.	Tela de detalhes por ID.
MealSummary	id, name, thumbUrl, área e país opcionais.	Cards dos filtros.
Category	id, name, thumbUrl, description.	Lista de categorias.
Area	name e country opcional.	Filtro por origem.
Ingredient	id, name, description, thumbUrl e type opcionais.	Filtro por ingrediente.
IngredientMeasure	name e measure.	Par ingrediente/medida da receita.

Para ingredientes, uma **List<IngredientMeasure>** preserva a ordem da API e evita perder itens se algum nome se repetir.

C. Serviço HTTP em Flutter (**MealService**)

O serviço existente é responsável somente pela comunicação com a API. A tela chama o serviço; o serviço faz o GET, decodifica o JSON e transforma os itens em **Meal**.

```
Future<List<Meal>> getFeaturedMeals() async {  
  final uri = Uri.parse(  
    'https://www.themealdb.com/api/json/v1/1/search.php?s=chicken',  
  );  
  final response = await http.get(uri);  
  if (response.statusCode != 200) {  
    throw MealServiceException('Erro ao buscar receitas');  
  }  
  final body = jsonDecode(response.body) as Map<String, dynamic>;  
  final mealsJson = body['meals'] as List<dynamic>;  
  if (mealsJson == null) return [];  
  return mealsJson  
    .map((item) => Meal.fromJson(item as Map<String, dynamic>))  
    .toList();  
}
```

Fluxo da aplicação:

Usuário abre o aplicativo

-> **HomePage** é carregada

-> **HomePage** chama **MealService**

-> **MealService** faz HTTP GET

-> **TheMealDB** responde JSON

-> **JSON** é convertido para **Meal**

-> **MealCard** mostra os dados

Documentação oficial da API:

<https://www.themealdb.com/api.php>